Lesson 7 — Cloud Storage and Distributed File Systems

INTE 22253 | Source: Lecture 07 "Cloud Storage and Distributed File Systems" (28 slides) | Also the core of Assignment 01 Part B (GFS, HDFS, S3 case study) | Builds on Lesson 4 (replication, consistency, CAP)

WHAT YOU WILL BE ABLE TO DO AFTER THIS LESSON

- Explain why storage must be distributed (scale, durability, elasticity).
- Compare **block, file and object storage** and pick one for a workload.
- Draw and explain the **GFS** and **HDFS** architectures, their consistency models and limitations, and compare them.
- Explain **Amazon S3**: buckets, objects, keys, durability, storage classes, consistency and access control.
- Compare **S3, EBS and EFS**, and discuss the four DFS design trade-offs.

1. Why cloud storage matters (slide 3)

Numbers on the slide: **181 ZB** of global data created/replicated by 2025 (IDC estimate); **99.999999999%** (11 nines) designed object durability for Amazon S3; distributed file systems like GFS built to reach **exabytes** (1 EB = 10^{18} bytes = a million terabytes).

Force	Why it pushes storage to be distributed
Scale	Single machines cannot hold or serve modern datasets
Durability	Hardware fails constantly at scale, so redundancy must be automatic
Elasticity	Storage must grow and shrink independently of compute

So storage designs go in one direction: **decoupled, distributed and fault-tolerant**.

2. The three cloud storage models (slides 4–9)

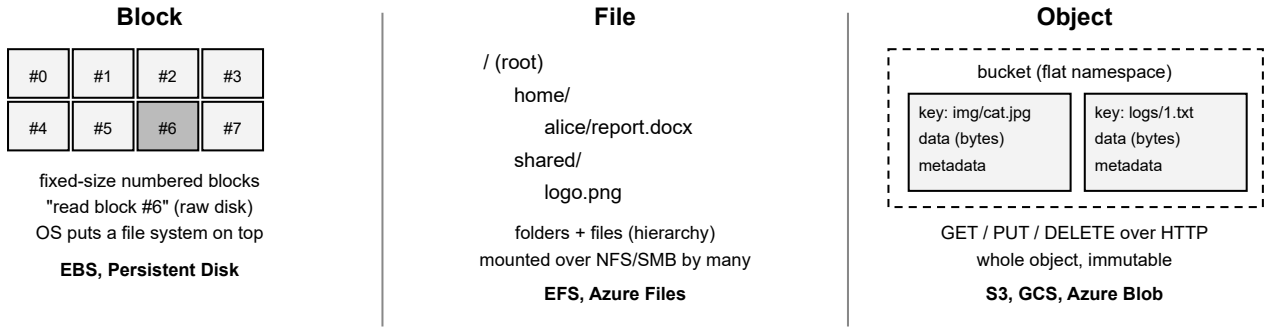


Figure 1 — Three storage models (slide 5). Each trades access granularity, latency and scale differently.

	Block storage (slide 6)	File storage (slide 7)	Object storage (slide 8)
Idea	Raw, addressable blocks — the closest cloud analog to a physical disk	A shared, hierarchical namespace accessed like a network drive	A flat namespace of immutable objects addressed by key over HTTP
How it works	Data split into fixed-size blocks, each independently addressable; the OS/app layers a file system on top (ext4, NTFS, XFS); random reads/writes at very low latency; typically attached to **one VM at a time** (AZ-bound)	Directories and files; accessed via **NFS, SMB/CIFS**; POSIX-like semantics (permissions, file locking); many clients mount it concurrently	Object = **data + rich metadata + unique key**; no true folders (prefixes simulate them); accessed via **REST API** (HTTP GET/PUT/DELETE); **immutable**: an update rewrites the whole object
Best for	Databases, boot volumes, transactional workloads; high IOPS, predictable	Shared home directories, CMS, lift-and-shift of legacy file-based apps	Backups, static assets, data lakes, media archives
Scale	Per volume, not automatically across a fleet	More elastic than block, less than object	Virtually unlimited (exabytes), no capacity planning
Examples	AWS EBS, Google Persistent Disk, Azure Managed Disks	AWS EFS, Azure Files, Google Filestore	Amazon S3, Google Cloud Storage, Azure Blob

Terms: IOPS = input/output operations per second. **POSIX** = the standard Unix file-system behaviour (open, read, write, permissions, locks). **NFS** (Unix/Linux) and **SMB/CIFS** (Windows) are network file-sharing protocols. **Data lake** = a huge store of raw data of all kinds kept for later analytics.

At a glance (slide 9)	Block	File	Object
Access unit	Block	File / directory	Whole object
Latency	Lowest	Low–medium	Higher
Scalability	Limited (per volume)	Medium	Massive (exabyte)
Metadata	Minimal	File-system attributes	Rich and custom
Best for	Databases, VMs	Shared file access	Unstructured, large-scale

Takeaway: block for low-latency structured I/O, file for shared hierarchical access, object for massive-scale unstructured data.

3. Google File System (GFS) (slides 11–13)

Aspect	Slide 11
Design philosophy	Assumes **hardware failure is the norm, not the exception** — reliability comes from software-level replication, not fault-tolerant hardware
Workload fit	Optimised for **large files (100 MB+)** with **sequential reads** and **append-heavy writes** — not general-purpose small-file access
Consistency	Deliberately **relaxed/weak**, traded for higher throughput on Google's workloads (crawling, indexing, MapReduce I/O)
Legacy	Its master/chunkserver design directly inspired HDFS and later systems

GFS architecture (slide 12)

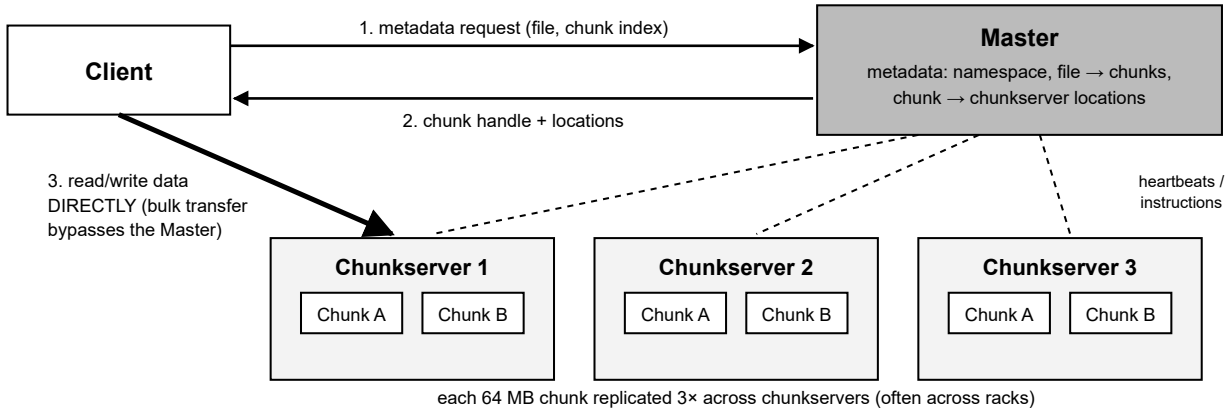


Figure 2 — A single Master holds metadata; chunkservers store replicated 64 MB chunks; data flows client ↔ chunkserver directly.

1. **Metadata request:** the client turns "byte offset in file" into a chunk index (offset ÷ 64 MB) and asks the Master for that chunk.
2. **Chunk location:** the Master replies with the chunk handle (ID) and which chunkservers hold its replicas. The client caches this.
3. **Data transfer:** the client reads/writes the data **directly** with a chunkserver. The Master never touches the data.

Replication: each 64 MB chunk is replicated **3× by default** across chunkservers (often across racks), so losing any single machine never loses data. **Key design choice:** separating the **metadata path** (through the Master) from the **data path** (client ↔ chunkserver) stops the Master becoming a throughput bottleneck, even though it remains a single point of coordination.

Worked example: chunks and replicas

A 1 GB (1,024 MB) log file in GFS: 1,024 ÷ 64 = **16 chunks**. With 3× replication: 16 × 3 = **48 chunk replicas**, using 3 GB of raw disk. To read byte offset 200 MB the client asks for chunk index ⌊200 ÷ 64⌋ = **3** (the 4th chunk, counting from 0). The Master stores only a small metadata record per chunk; the 1 GB of data never passes through it.

GFS consistency model and limitations (slide 13)

Consistency model

- **Relaxed** rather than strict: favours throughput over strict correctness guarantees.
- Concurrent writes to the same region may leave it **"consistent but undefined"**: all replicas hold the same bytes, but they may be a mix of fragments from several writers, so no single write is visible whole.
- **Record appends are at-least-once**: if an append fails on one replica the client retries, so **duplicates and padding** are possible.
- Applications must tolerate this (checksums, unique record IDs to drop duplicates).

Key limitations

- **Single Master** is a metadata bottleneck and historical single point of failure.
- Poor fit for **many small files** (all metadata in Master memory).
- Not for **low-latency random writes** or interactive apps.
- Tightly coupled to Google's workloads — **never released as a product**.

4. Hadoop Distributed File System (HDFS) (slides 14–17)

An **open-source** file system heavily inspired by GFS, built for the **Apache Hadoop** ecosystem. Originally built to support MapReduce; now underlies Spark, Hive and more. Widely deployed outside any single company, on commodity hardware, on-premises or in the cloud.

GFS term	HDFS term	Role
Master	**NameNode**	Holds metadata (namespace, block locations)
Chunkserver	**DataNode**	Stores the actual data blocks
Chunk (64 MB)	**Block (128 MB default)**	Unit of storage and replication; 3× by default

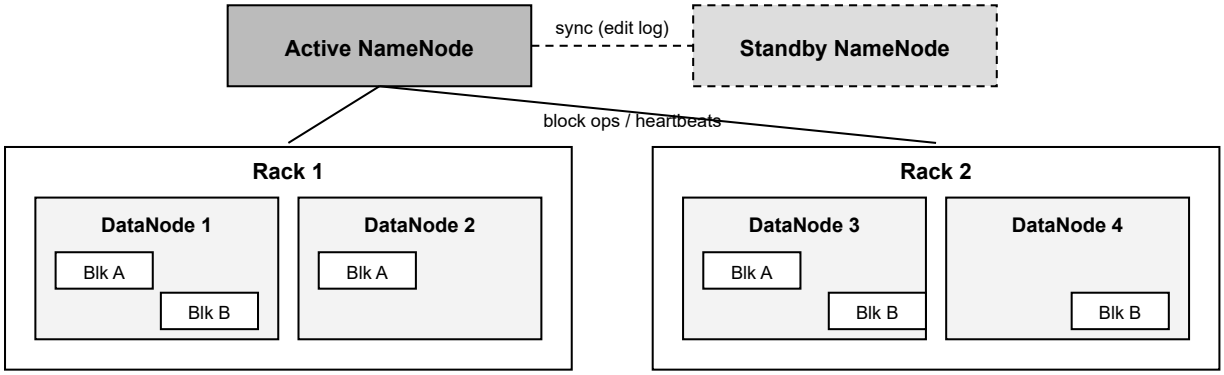


Figure 3 — HDFS (slide 15): Active/Standby NameNodes for high availability; DataNodes with rack-aware replicas (block A: 2 copies on rack 1, 1 on rack 2).

- **High availability:** an **Active** NameNode serves requests; a **Standby** NameNode keeps in sync through the shared **edit log** (the journal of every metadata change) and takes over if the active fails. This fixes the original single point of failure.
- **Rack awareness:** HDFS deliberately places replicas across racks (e.g. **2 copies on one rack, 1 on another**), so a whole rack or its top-of-rack switch failing never loses data — a refinement beyond GFS's original design. Two on the same rack keeps write traffic cheaper; one elsewhere protects against rack loss.

HDFS features and the small-file problem (slide 16)

Feature	Explanation
Write-once, read-many	Optimised for batch analytics (MapReduce, Spark, Hive), not transactional updates. Files are written, closed, then read many times.
128 MB default block size	Large blocks minimise metadata and suit sequential scans of huge datasets.
Erasure coding (HDFS 3.x)	**Reed-Solomon** coding stores data blocks plus a few calculated **parity** blocks; lost blocks can be rebuilt from the rest. Storage overhead drops to **~1.5×** instead of 3×, at extra CPU cost to rebuild.
Small-file problem	Every file's and block's metadata lives in **NameNode RAM**; millions of small files can exhaust memory and hurt performance.

Worked example: replication vs erasure coding, and small files

Case	Calculation
Store 600 MB with 3× replication	600 × 3 = **1,800 MB** raw disk; survives losing any 2 copies
Same 600 MB with Reed-Solomon RS(6,3): 6 data blocks + 3 parity blocks	Raw = 600 × 9/6 = **900 MB (1.5×)**; survives losing any 3 of the 9 blocks, but rebuilding needs CPU
1 GB stored as one file vs as 10,000 files of 100 KB	One file = 8 blocks (128 MB) ⇒ ~9 metadata objects. 10,000 small files = 10,000 file entries + 10,000 blocks ⇒ ~20,000 objects in NameNode RAM for the same data: about **2,000×** the metadata.

GFS vs HDFS (slide 17)

	GFS	HDFS
Origin	Internal Google system (2003 paper)	Open-source, Apache Hadoop project
Metadata node	Single Master	NameNode (HA Active/Standby pairs)
Default chunk/block size	64 MB	128 MB
Replication	3× (default)	3× default, or erasure coding (3.x)
Placement awareness	Basic	Explicit rack awareness
Primary workload	Google internal batch (search, indexing)	General-purpose Hadoop ecosystem (Spark, Hive)

5. Amazon S3 (slides 18–21)

A **managed object storage service** — not a traditional distributed file system.

Concept	Meaning
Bucket	Top-level container for objects; **globally unique name**; bound to one AWS region
Object	The unit of storage: data plus metadata, addressed by a key inside a bucket
Key	The unique identifier of an object in a bucket. **Flat namespace**: "folders" are just key prefixes (`photos/2026/cat.jpg` is one key)
REST API	All access (console, CLI, SDKs) goes through HTTP(S) calls: GET, PUT, DELETE, LIST

```
PUT    https://my-bucket.s3.amazonaws.com/photos/2026/cat.jpg   # upload (whole object)
GET    https://my-bucket.s3.amazonaws.com/photos/2026/cat.jpg   # download
GET    https://my-bucket.s3.amazonaws.com/?prefix=photos/2026/  # LIST keys with this prefix
DELETE https://my-bucket.s3.amazonaws.com/photos/2026/cat.jpg
```

Durability and replication (slide 19)

- **99.999999999% ("11 nines")** designed durability per year. Intuition: store 10 million objects and you would expect to lose one about once every 10,000 years.
- **Multi-AZ by default:** objects stored redundantly across multiple Availability Zones in the region; a whole AZ can fail without data loss.
- **No server management:** replication, hardware failures and scaling are handled internally; users never provision nodes.
- **Versioning**: keeps every version of an object (protects against accidental overwrite/delete). **Cross-Region Replication (CRR)**: asynchronously copies objects to a bucket in another region. **Object Lock**: **WORM** (write once, read many) protection for compliance (retention periods, legal holds).

Durability vs availability: durability = the data is not lost; availability = you can reach it right now. S3 can be briefly unreachable yet still durable.

S3 storage classes (slide 20)

Storage class	Access pattern	Retrieval time	Relative cost
S3 Standard	Frequent access	Milliseconds	Highest
S3 Intelligent-Tiering	Unknown/changing	Milliseconds	Auto-optimised
S3 Standard-IA (Infrequent Access)	Infrequent access	Milliseconds	Lower storage, retrieval fee
S3 One Zone-IA	Infrequent, re-creatable data	Milliseconds	Lower (single AZ)
Glacier Instant Retrieval	Rarely accessed archives	Milliseconds	Very low
Glacier Flexible Retrieval	Archival	Minutes–hours	Very low
Glacier Deep Archive	Long-term compliance archive	Hours (up to 12 h)	Lowest

Lifecycle policies move objects between classes automatically as they age. Example: website images in Standard → after 30 days to Standard-IA → after 1 year to Glacier Deep Archive for a 7-year legal retention.

Consistency and access control (slide 21)

Consistency model

- **Strong read-after-write consistency** for all requests **since December 2020**.
- Applies to new-object PUTs, overwrite PUTs and DELETEs alike.
- Replaced the earlier **eventual** consistency for overwrites/deletes.
- Simplifies apps: no stale reads after a write.

Security & access control

- **IAM policies**: which users/roles can act on which buckets.
- **Bucket policies**: resource-level rules (allow/deny by IP, VPC).
- **Encryption at rest**: SSE-S3 (S3-managed keys), SSE-KMS (AWS Key Management Service keys), SSE-C (customer-provided keys).
- **Block Public Access** guards against accidental public exposure.

6. The AWS storage ecosystem (slides 22–26)

Service	Type	What it is (slide 23)
EBS	Block	Persistent block volumes for EC2 instances
EFS	File	Elastic NFS file system, multi-AZ
S3	Object	Scalable object storage, 11 nines durability
Glacier	Archive	Low-cost long-term archival (an S3 class)
FSx	File	Managed Windows / Lustre / NetApp / OpenZFS file systems
Storage Gateway	Hybrid	Bridges on-premises environments to AWS storage
Snow Family	Transfer	Physical devices shipped to you for offline bulk data migration
Backup	Management	Centralized backup policy across AWS services

EBS: Elastic Block Store (slide 24)

Volume type	Best for	Key spec
gp3 (General Purpose SSD)	Boot volumes, dev/test, most workloads	3,000 IOPS / 125 MB/s baseline, scalable independently
io2 Block Express	High-performance databases (Oracle, SAP)	Up to 256,000 IOPS, sub-ms latency, 99.999% durability
st1 (Throughput HDD)	Big data, log processing, streaming	Low cost, high throughput, not for random I/O
sc1 (Cold HDD)	Infrequently accessed data	Lowest cost per GB of EBS

EBS is **bound to a single AZ** — snapshot it to move across AZs/regions. **Point-in-time snapshots** are stored durably in S3, **incremental** after the first (only changed blocks). Volumes can be resized and their type changed without downtime.

EFS: Elastic File System (slide 25)

A fully managed, elastic **NFSv4** file system shared across many instances and AZs. **Storage classes:** EFS Standard (frequent, multi-AZ), EFS Standard-IA (auto-tiers infrequent files, cheaper), EFS One Zone / One Zone-IA (single AZ, cheaper); lifecycle management moves files between classes. **Throughput modes: bursting** (throughput scales with storage size, with burst credits), **provisioned** (fixed throughput regardless of size), **elastic** (scales up/down automatically with the workload). Grows and shrinks automatically — no capacity planning.

S3 vs EBS vs EFS (slide 26)

	Amazon S3	Amazon EBS	Amazon EFS
Storage type	Object	Block	File
Attach point	Accessed over network/API	One EC2 instance at a time*	Many instances, multi-AZ
Scalability	Virtually unlimited	Fixed size (resizable)	Elastic, grows automatically
Latency	Higher (network/API calls)	Lowest (near-local disk)	Low–medium (network FS)
Typical use	Backups, data lakes, static assets	Databases, boot volumes	Shared content, home directories
Durability model	11 nines, multi-AZ automatic	Single AZ; snapshots to S3	Multi-AZ (Standard class)

*io2 Block Express Multi-Attach lets one io2 volume attach to several instances in limited scenarios.

Worked example: pick the storage

Need	Choice	Why
MySQL database on an EC2 instance	EBS (gp3, or io2 for heavy load)	Low-latency random block I/O
Ten web servers must share the same uploaded files	EFS	Many instances, many AZs, one file system
Millions of product photos served to a website	S3 Standard	Unlimited scale, HTTP access, cheap
Bank records kept 7 years, almost never read	S3 Glacier Deep Archive + Object Lock	Lowest cost; WORM for compliance
Nightly MapReduce over 50 TB of logs on-premises	HDFS	Large sequential batch reads, open source, commodity cluster

7. DFS design trade-offs (slide 27)

Trade-off	Explanation
1. Consistency vs availability	Strong consistency costs latency/availability during partitions; eventual consistency improves availability but risks stale reads. GFS/HDFS are relaxed; S3 is strong (CAP, Lesson 4).
2. Metadata: centralized vs distributed	A single master (GFS) is simple but a scaling/SPOF risk; distributed metadata (Ceph, HDFS Federation) scales further but needs **consensus protocols** (Paxos/Raft) so the metadata servers agree.
3. Replication strategy	Full 3× replication is simple and fast to recover but costly; erasure coding cuts storage to ~1.5× at the cost of CPU-heavy rebuilds.
4. Block size and access pattern	Large blocks (64–128 MB) suit sequential big-file workloads; small blocks suit random access but blow up metadata overhead.

Key takeaways (slide 28): (1) match storage type to access pattern; (2) GFS and HDFS pioneered the master/worker DFS pattern with relaxed consistency for large sequential batch work; (3) S3 is a different design point: managed object storage trading file-system semantics for massive scale and near-zero operations; (4) AWS pairs S3 with EBS (block) and EFS (file), each with its own durability, scaling and cost model; (5) storage design is a continuous set of trade-offs: consistency vs availability, replication cost vs durability, latency vs throughput.

8. Exam-style questions with model answers

Q1. Compare block, file and object storage with an example and a use case of each. (9 marks)
Block: data in fixed-size independently addressable blocks; the OS puts a file system on top; lowest latency, random read/write; usually attached to one VM; scales per volume; for databases and boot volumes (AWS EBS). File: hierarchical directories and files accessed over NFS/SMB with POSIX semantics (permissions, locking); many clients mount it at once; medium scalability; for shared home directories, CMS, lift-and-shift apps (AWS EFS). Object: flat namespace of immutable objects (data + rich metadata + unique key) accessed by REST HTTP calls; higher latency but virtually unlimited scale; for backups, static assets, data lakes, media archives (Amazon S3).

Q2. Draw and explain the GFS architecture, including how a read works. (8)
(Draw Figure 2.) A single Master stores all metadata (namespace, file-to-chunk mapping, chunk locations); many chunkservers store 64 MB chunks, each replicated 3× across chunkservers/racks. Read: the client computes the chunk index from the byte offset and asks the Master; the Master returns the chunk handle and replica locations; the client then reads the data directly from a chunkserver. Separating the metadata path from the data path keeps the Master from becoming a throughput bottleneck.

Q3. Describe the GFS consistency model and two limitations. (5)
GFS uses relaxed consistency for throughput: concurrent writes can leave a region consistent but undefined (same bytes on all replicas but mixed fragments), and record appends are at-least-once, so duplicates and padding can

occur; applications use checksums and unique IDs. Limitations: the single Master is a metadata bottleneck and single point of failure; poor for many small files; not for low-latency random writes; never released as a product.

Q4. How does HDFS improve on GFS? (6)
HDFS is open source (Apache Hadoop); uses a NameNode/DataNode design with 128 MB blocks (less metadata); provides high availability with Active/Standby NameNodes synchronised by the edit log, removing the SPOF; uses explicit rack awareness (e.g. two replicas on one rack, one on another) to survive rack failures; and supports erasure coding in 3.x, cutting overhead from 3× to about 1.5×.

Q5. What is the small-file problem in HDFS? (3)
All file and block metadata is held in NameNode RAM, so millions of small files create millions of metadata entries that can exhaust NameNode memory and slow it down, even when the total data is small.

Q6. Compare 3× replication with erasure coding. (4)
3× replication keeps three full copies: simple, fast recovery, but 200% storage overhead. Erasure coding (Reed-Solomon) stores data plus parity blocks and rebuilds lost blocks mathematically: ~1.5× storage, but rebuilding costs CPU and is slower.

Q7. Explain buckets, objects and keys in S3, and how S3 achieves durability. (6)
A bucket is a globally uniquely named, region-bound container; an object is data plus metadata; a key uniquely identifies an object in a bucket, in a flat namespace where "folders" are key prefixes; all access is through the REST API. S3 is designed for 11 nines durability by storing objects redundantly across multiple AZs automatically, with versioning, cross-region replication and Object Lock for further protection.

Q8. What consistency model does S3 provide today? Why does it matter? (3)
Strong read-after-write consistency for all PUTs (new and overwrite) and DELETEs since December 2020, replacing eventual consistency for overwrites and deletes. Applications no longer need to handle stale reads after a write.

Q9. Recommend S3 storage classes for: website images, unknown access pattern, 7-year compliance archive. (3)
S3 Standard (frequent, milliseconds); S3 Intelligent-Tiering (unknown/changing pattern, auto-optimised); Glacier Deep Archive (lowest cost, retrieval up to 12 h), ideally with Object Lock.

Q10. Compare S3, EBS and EFS. (6)
S3: object, accessed by API, unlimited scale, higher latency, backups/data lakes, 11 nines multi-AZ. EBS: block, attached to one EC2 instance, fixed but resizable, lowest latency, databases/boot volumes, single AZ with snapshots to S3. EFS: file (NFS), shared by many instances across AZs, elastic, low-medium latency, shared content/home directories, multi-AZ in Standard class.

Q11. Classify GFS, HDFS and S3 as block, file or object storage and justify. (Assignment Part B) (6)
GFS and HDFS are distributed **file** systems: they expose a hierarchical namespace of files and directories, internally split into large chunks/blocks stored on many servers. S3 is **object** storage: a flat namespace of immutable objects with metadata, addressed by key via a REST API, with no real directories or in-place updates.

Q12. Discuss four design trade-offs when building a distributed file system. (8)
Consistency vs availability (strong costs latency/availability under partitions; eventual risks stale reads). Centralized vs distributed metadata (a single master is simple but a bottleneck/SPOF; distributed metadata scales but needs Paxos/Raft consensus). Replication strategy (3× is simple and quick to recover but costly; erasure coding saves space but costs CPU on rebuild). Block size (large blocks suit sequential big files and small metadata; small blocks suit random access but inflate metadata).

9. Common mistakes

Mistake	Correct idea
S3 has real folders	Flat namespace; "folders" are key prefixes.
Objects can be edited in place	Objects are immutable; an update rewrites the whole object.
Data in GFS flows through the Master	Only metadata goes through the Master; data goes client ↔ chunkserver.
GFS block size 128 MB, HDFS 64 MB	GFS 64 MB chunks; HDFS 128 MB blocks.
S3 is eventually consistent	Strong read-after-write since December 2020.
Durability = availability	Durability = not losing data; availability = being reachable now.
EBS can be shared by many instances like EFS	EBS attaches to one instance (except io2 Multi-Attach); EFS is shared.
EBS survives an AZ failure automatically	EBS is AZ-bound; use snapshots (stored in S3) to recover elsewhere.
HDFS is good for millions of tiny files	Small-file problem: NameNode memory.
Erasure coding is free	Saves storage (~1.5×) but costs CPU for rebuilds.

Cheat sheet

	Block	File	Object
Unit	Block	File/dir	Whole object
Access	Raw disk, FS on top	NFS / SMB, POSIX	REST GET/PUT/DELETE
Latency	Lowest	Low–med	Higher
Scale	Per volume	Medium	Exabytes
Metadata	Minimal	FS attrs	Rich, custom
Use	DBs, VMs, boot	Shared dirs, CMS	Backups, data lakes, media
AWS	EBS	EFS, FSx	S3, Glacier

	GFS	HDFS
Origin	Google 2003, internal	Apache Hadoop, open
Metadata	Single Master	NameNode Active/Standby (edit log)
Data	Chunkservers	DataNodes
Unit	64 MB chunk	128 MB block
Replicas	3×	3× or erasure coding (~1.5×)
Placement	Basic	Rack aware (2 + 1)
Consistency	Relaxed; GFS appends at-least-once ("consistent but undefined")	
Weakness	Master/NameNode memory: small files; not for low-latency random writes	
Read	Ask master for location → read data directly from server	

Amazon S3	
Concepts	Bucket (unique, region) · object (data + metadata) · key (flat, prefixes) · REST
Durability	11 nines, multi-AZ; versioning, CRR, Object Lock (WORM)
Consistency	Strong read-after-write since Dec 2020
Security	IAM, bucket policies, SSE-S3/KMS/C, Block Public Access
Classes	Standard → Intelligent-Tiering → Standard-IA → One Zone-IA → Glacier Instant → Glacier Flexible (min–h) → Deep Archive (≤12 h); lifecycle policies

EBS / EFS	
EBS	gp3 (3,000 IOPS), io2 (256K IOPS, DBs), st1 (throughput HDD), sc1 (cold). One AZ; incremental snapshots to S3; resize live
EFS	Managed NFSv4, many instances + AZs; Standard / IA / One Zone; bursting / provisioned / elastic throughput
Others	FSx, Storage Gateway (hybrid), Snow (offline transfer), Backup

Trade-offs	
1	Consistency vs availability
2	Central vs distributed metadata (Paxos/Raft)
3	3× replication vs erasure coding
4	Large vs small blocks